

Week 6: Propaganda Detection

This week we will be looking at the propaganda detection task (Da San Martino 2019) and developing a baseline model for technique classification.

We will be working with an adapted version of the dataset from the paper. In particular, I have

- reduced the number of propaganda techniques
- randomly sampled sentences labelled with a particular technique
- randomly sampled sentences (from the same original articles) which do not contain propaganda
- reformatted the data so that the snippets can be "read" in the context of the sentence by inserting \<BOS> and \<EOS> tags.

Let's load it up in a pandas dataframe so that we can look at some examples to illustrate that last point.

```
In [1]: import os
parentdir = "/Users/juliewe/Dropbox/teaching/AdvancedNLP/2025/2025_content/week6/lab6/"
train_file= "propaganda_train.tsv"
train_path=os.path.join(parentdir,train_file)
```

```
In [2]: import pandas as pd
train_df=pd.read_csv(train_path,delimiter="\t",quotechar='|')
train_df.head(20)
```

```
Out[2]:
```

	label	tagged_in_context
0	not_propaganda	No, <BOS> he <EOS> will not be confirmed.
1	not_propaganda	This declassification effort <BOS> won't make ...
2	flag_waving	"The Obama administration misled the <BOS> Ame...
3	not_propaganda	"It looks like we're capturing the demise of t...
4	not_propaganda	<BOS> Location: Westerville, Ohio <EOS>
5	loaded_language	Hitler <BOS> annihilated <EOS> 400,000 Germans...
6	not_propaganda	A federal judge on Monday ordered U.S. immigra...
7	not_propaganda	<BOS> Kirstjen Nielsen (@SecNielsen) <EOS> Nov...
8	doubt	As noted above, at this point literally every ...
9	not_propaganda	Britain doesn't need more hate even just for a...
10	name_calling,labeling	His account was suspended for violating Twitte...
11	not_propaganda	Ironically, even in doing this he is <BOS> lik...
12	appeal_to_fear_prejudice	A couple of seemingly unrelated events this pa...
13	not_propaganda	During the term of the Assassination <BOS> Rec...
14	repetition	Trump has expelled <BOS> more <EOS> Russian di...
15	not_propaganda	President Trump ordered that the relevant docu...
16	not_propaganda	The newspaper added that Manafort's visit was ...
17	not_propaganda	"We are now doing a <BOS> recount, so <EOS> we...
18	doubt	<BOS> But who authorized this strategic commit...
19	causal_oversimplification	Outside of specialists and seekers, <BOS> the ...

We can see in the cell above that there are 2 columns. The first column is the label i.e., the propaganda technique or the label "not_propaganda". The 2nd column contains the text. Within the

text we can see the special \<BOS> and \<EOS> tags which indicate the propaganda snippet.

Let's have a look at some examples from the **loaded_language** class.

```
In [3]: train_df[train_df["label"]=="loaded_language"]
```

```
Out[3]:
```

	label	tagged_in_context
5	loaded_language	Hitler <BOS> annihilated <EOS> 400,000 Germans...
21	loaded_language	We've caught on to his <BOS> sneaky <EOS> syno...
49	loaded_language	This is why it is so <BOS> heinous <EOS>."
106	loaded_language	Here is the <BOS> dirty <EOS> secret that is n...
124	loaded_language	She offered no evidence — nor was asked for an...
...	...	...
2447	loaded_language	The ADL described the video as "<BOS> venomous...
2458	loaded_language	What started after the 2016 election as a redu...
2487	loaded_language	Its superior had made the <BOS> fatal <EOS> er...
2496	loaded_language	Ridicule is a <BOS> vicious and potent <EOS> w...
2519	loaded_language	But this move demonstrates—if more evidence we...

161 rows x 2 columns

In the first loaded_language example (row 5 of the original dataframe), we can see that there is a single word *annihilated* which is between \<BOS> (**beginning of span**) and \<EOS> (**end of span**).

Exercise 1: Exploratory Data Analysis

Write code and plot appropriate graphs to visualise each of the following questions.

- How many samples are there for each class?
- What is the average length of sentence for each class?
- What is the average length of propaganda snippet for each class?

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

Exercise 2: Sentence Level Binary classification

Build a simple classifier (e.g., Naïve Bayes or Logistic Regression) which can take a sentence and predict whether it contains propaganda or not. Things you will need to think about

- making a binary "propaganda" or "not_propaganda" label
- splitting the data into training and validation
- making a bag-of-words representation of each sentence. This could be a dictionary where the keys are the words and the values are the frequencies within the sentence.
- the implementation of the classifier itself. You are not expected to build this yourself. A good one to use would be the multinomialNB classifier in scikit-learn

It's worth thinking about the input format expected by the classifier before pre-processing the data.
Code to import and use the scikit-learn multinomialNB classifier is below for consideration

```
In [4]: #This gives us some random toy data.
#In this toy data there are 10 data points (e.g., sentences)
# each sentence is represented as a vector of 100 values
# each value could be the frequency of a particular word in the vocab. The max frequency
# these are stored in X
# there are 2 possible labels (0 and 1) which are stored in Y

import numpy as np
rng=np.random.RandomState()
X = rng.randint(6,size=(10,100))
Y = rng.randint(2, size=10)

print(X)
print(Y)
```

```
[[0 1 1 2 4 4 1 4 1 0 2 3 1 4 1 1 3 4 2 4 3 1 0 2 2 4 4 0 1 5 4 2 5 5 1 5
 1 2 5 2 4 2 1 0 4 3 0 0 1 3 0 5 5 3 2 5 0 4 5 1 4 5 5 4 5 2 5 3 4 4 5 4
 2 1 2 4 3 5 2 1 1 2 5 5 1 2 5 5 2 5 1 4 0 0 5 2 4 4 4 3]
[4 2 5 2 2 5 3 0 4 3 5 4 4 2 4 4 4 4 1 1 2 0 4 1 5 5 3 2 3 2 5 1 2 4 1
 1 4 5 3 1 4 2 3 0 0 1 2 3 3 5 1 5 5 0 3 0 0 0 4 5 4 0 5 1 5 5 4 3 4 4 3
 4 2 2 1 2 5 3 0 2 2 2 5 1 1 1 0 3 4 3 5 1 0 2 0 3 4 5 3]
[3 4 2 3 3 5 0 0 4 3 3 5 5 5 0 0 1 5 3 0 4 3 4 4 5 4 5 1 2 1 5 4 5 3 3 2
 0 0 3 2 4 4 4 1 5 3 4 1 4 2 1 5 2 3 1 1 3 4 0 3 0 2 0 5 5 2 0 3 5 0 3 3
 4 2 1 2 0 4 2 4 0 4 5 0 3 4 2 1 2 0 1 3 4 0 3 5 1 0 5 2]
[4 4 0 4 1 4 3 1 3 1 1 0 4 5 3 5 4 1 2 4 2 3 5 3 2 0 4 4 1 1 0 3 0 1 3 2
 4 5 2 2 5 4 1 3 1 4 5 0 5 1 4 0 3 4 5 1 1 1 1 5 4 1 2 3 1 4 1 3 2 3 4 3
 2 5 1 5 2 3 1 0 0 4 5 5 3 4 3 4 0 0 1 3 0 0 0 0 0 1 3 4]
[4 1 5 1 4 3 4 0 3 5 2 2 3 4 2 3 4 0 2 5 1 3 0 4 4 3 1 5 0 2 3 4 2 1 2 3
 5 4 4 4 1 1 5 4 4 1 1 4 4 2 1 4 4 0 4 1 2 3 0 4 3 1 2 3 2 0 4 4 2 4 2
 1 2 3 1 5 4 1 1 0 4 5 2 1 2 2 3 3 0 3 1 3 1 4 5 5 0 0 5]
[3 5 4 1 3 5 4 3 4 4 3 5 0 0 3 3 2 4 2 4 3 0 0 2 1 1 4 0 2 3 4 3 4 4 1 4
 0 4 4 1 2 1 1 3 4 3 4 4 5 3 3 0 5 5 0 3 1 5 0 1 4 5 1 0 2 1 3 2 5 2 0 1
 3 2 2 4 4 1 0 2 0 3 5 0 4 4 4 0 3 1 2 4 1 2 1 0 0 1 1 1]
[1 3 5 5 4 5 0 5 4 2 2 5 3 5 3 1 3 0 2 3 1 1 2 0 2 3 2 2 4 4 2 5 5 4 0 4
 1 4 2 2 1 0 0 4 1 5 2 2 4 3 2 5 5 3 5 0 4 3 2 1 2 3 5 1 5 2 0 5 2 4 3 3
 1 5 1 0 1 4 4 3 4 2 3 4 1 1 2 5 1 1 5 5 4 2 3 2 0 1 0 4]
[4 3 3 3 1 5 3 0 3 1 5 1 1 5 5 5 4 1 1 1 3 2 2 1 1 2 1 2 1 1 5 2 4 0 3 2
 0 3 0 2 3 3 1 1 3 5 0 3 0 3 0 3 0 4 2 5 5 5 5 0 0 5 5 2 3 0 0 5 3 5 5 5
 2 0 5 5 5 1 3 5 5 5 0 5 2 5 2 5 1 5 2 4 0 2 1 3 5 2 5 5]
[4 0 1 3 0 2 5 0 1 5 2 0 1 2 1 1 4 4 2 3 0 0 3 4 0 4 5 0 1 0 3 4 0 3 1 4
 2 3 5 4 5 4 5 0 4 4 0 4 0 3 5 5 3 3 0 3 0 2 4 1 3 5 0 1 2 1 4 0 5 2 0 0
 2 0 4 4 3 4 0 1 4 2 0 4 3 0 3 2 5 3 0 1 5 2 1 1 1 2 2 4]
[1 4 5 0 1 4 1 4 4 5 3 1 4 4 5 0 4 5 4 2 0 4 3 3 5 4 1 5 0 1 5 5 2 3 5 1
 4 1 0 1 5 3 5 2 0 4 2 1 0 1 4 3 2 2 0 0 1 5 4 0 2 1 4 1 4 5 4 4 1 3 5 3
 5 2 3 3 0 5 2 0 2 2 2 1 3 3 4 4 4 0 1 5 4 5 0 5 3 2 3 4]]
[1 1 1 1 1 0 1 0 0 1]
```

```
In [5]: #we can give this as input to the MultinomialNB classifier using the fit method
from sklearn.naive_bayes import MultinomialNB

classifier=MultinomialNB()
classifier.fit(X,Y)
```

```
Out[5]: ▼ MultinomialNB
MultinomialNB()
```

```
In [6]: #we can predict the value for any datapoint
#here we are making up some more random points with random labels so I wouldn't expect

X = rng.randint(6,size=(5,100))
Y = rng.randint(2, size=5)
print(Y)
classifier.predict(X)

[0 1 0 0 1]
```

```
Out[6]: array([1, 1, 1, 1, 1])
```

So imagine you have sentence representations and labels as follows. We need to generate vectors for them where each column corresponds to a particular word in the vocabulary

```
In [7]: toy_training_data=[({"everyone":1,"hates":1,"vectors":1},1),({"vectors":1,"are":1,"useful":1,"hates":1,"vectors":1},1)]
Xdicts,Y=zip(*toy_training_data)
```

We could write some code to turn the Xdicts into vectors (you first need to work out what the vocab is and assign an index to each vocab item). Or we can use scikit-learn's CountVectorizer directly on the texts.

```
In [8]: toy_training_data=[("everyone hates vectors",1),("vectors are useful",0)]
Xsents,Y=zip(*toy_training_data)
```

```
In [9]: from sklearn.feature_extraction.text import CountVectorizer
vectorizer=CountVectorizer()
vectorizer.fit(Xsents)

# Printing the identified Unique words along with their indices
print("Vocabulary: ", vectorizer.vocabulary_)

# Encode the sents
Xvectors = vectorizer.transform(Xsents)

print(Xvectors)
```

```
Vocabulary:  {'everyone': 1, 'hates': 2, 'vectors': 4, 'are': 0, 'useful': 3}
(0, 1)      1
(0, 2)      1
(0, 4)      1
(1, 0)      1
(1, 3)      1
(1, 4)      1
```

```
In [10]: classifier=MultinomialNB()
classifier.fit(Xvectors,Y)
```

```
Out[10]: ▼ MultinomialNB
MultinomialNB()
```

```
In [11]: classifier.predict(Xvectors[1])
```

```
Out[11]: array([0])
```

What happens if some new sentences have words not in the vocabulary? We can see here that they are just ignored by the vectorization process – this is fine as they are not going to help the classifier as they are unknown

```
In [12]: toy_test_data=[("everyone hates useful vectors",1),("vectors are really useful",0)]
Xsents,Y=zip(*toy_test_data)
testVectors=vectorizer.transform(Xsents)

print(testVectors)
```

```
(0, 1)      1
(0, 2)      1
(0, 3)      1
(0, 4)      1
(1, 0)      1
(1, 3)      1
(1, 4)      1
```

```
In [13]: classifier.predict(testVectors)
```

```
Out[13]: array([1, 0])
```

Lets go back to a modified version of the exercise which assumes we are going to use CountVectorizer and MultinomialNB

Build a MultinomialNB classifier which can take a sentence and predict whether it contains propaganda or not. Things you will need to think about

- making a binary "propaganda" or "not_propaganda" label
- splitting the data into training and validation
- making a bag-of-words representation of each sentence using CountVectorizer
- Training a MultinomialNB classifier on the training data and evaluating it on the validation data

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

Exercise 3: Snippet level binary classification

Repeat exercise 2 but train and test on propaganda snippets rather than whole sentences.

```
In [ ]:
```

Exercise 4: Mixing snippets and sentence

- How well does your sentence level classifier work on snippets?
- How well does snippet level classifier work on sentences?
- What about if you train on both sentences and snippets?

```
In [ ]:
```

Extensions

- 1) Can you use cross-validation to evaluate your classifiers rather than a single training/development split
- 2) Can you use a different type of classifier and / or feature representation. E.g., logistic regression where the feature values are tf-idf values rather than frequencies?
- 3) Can you carry out multi-class classification to identify the propaganda technique used?
- 4) Does it help to first use your binary classifier to decide whether there is any propaganda or not?

```
In [ ]:
```